



ADVANTEST CORPORATION

*ACS Real-Time
Data Infrastructure (RTDI)
ONEAPI User Guide*

This version includes:

ACS Nexus v3.2.0

Legal Notices

All rights reserved. All text and figures included in this publication are the exclusive property of Advantest Corporation. Reproduction of this publication in any manner without the written permission of Advantest Corporation is prohibited. Information in this document is subject to change without notice.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this document may be photocopied, reproduced, or translated to another language without prior written consent of Advantest Corporation.

Trademarks and Registered Trademarks

- ADVANTEST is a trademark of Advantest Corporation.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

1.1 OneAPI Python SDK

OneAPI is the standard bi-directional communication interface that enable containerized/non-containerized applications to consume real time data from (and send control command to) ACS Nexus. An application developer can use OneAPI to develop an application to consume the real time data and send control instructions during production testing. ACS Nexus invokes a callback function of the containerized application per event.

In this version of OneAPI, only C++ SDK and Python SDK are supported. This section describes Python SDK usage.

1.1.1 General Information

Package Contents

OneAPI Python SDK provides programming interfaces for the user to develop applications. All contents (see below) are packaged in a tar.gz file. Note that the library Python 3.9, 3.10, and 3.11 library files are in different directories.

```
oneAPI_py3.9 (oneAPI_py3.10 and oneAPI_py3.11 have the same file structure)
|-- bin
|   |-- AdvantestLogging.py
|   |-- oneapi_DFF.py
|   |-- oneapi.py
|   |-- liboneAPI.so
|   |-- main.py
|   |-- requirements.txt
|   |-- sample.py
|-- build_base_image.sh
|-- centos.Dockerfile.example (for py3.9 only)
|-- ubuntu.Dockerfile.example
```

Table Error! No text of specified style in document.-1. OneAPI Python Package Content Descriptions

Content	Description
bin	This folder contains all library files provided by OneAPI for Python. Developers need to link these files and include them in the application release-package or image.
build_base_image.sh	This script provides a reference for users on how to build an image with Dockerfile.
centos.Dockerfile.example ubuntu.Dockerfile.example	NOTE: centos.Dockerfile.example is for python 3.9 only. These files provide a reference for users on how to build a OneAPI Application image based on the base image. Users need to rewrite Dockerfile according to their actual situation. Refer to the comments in this file for specific rewriting methods.

Environment Requirements

Users can develop containerized applications running on the ACS Edge Server or containerized/non-containerized applications running on a test floor server that is based on OneAPI Python SDK. For both scenarios, specific environment conditions are required, as noted below.

Application Development Environment	
Python 3.9 SDK	
Supported OS (any of)	• Red Hat 7 • Red Hat 9 • CentOS 7 • Ubuntu 22.04
Necessary Software	• Python 3.9 • Python module: jsonschema
Python 3.10 SDK	
Supported OS	• Ubuntu 22.04
Necessary Software	• Python 3.10 • Python module: jsonschema
Python 3.11 SDK	
Supported OS	Ubuntu 22.04
Necessary Software	• Python 3.11 • Python module: jsonschema

Application Operating Environment
A successful connection at least needs to ensure that the network between Application Operating environment and ACS Nexus operation environment is enabled, and other dependent guarantees, such as ports availability, firewall permission, etc.

1.1.2 Usage Scenario

Real Time Test Cell Data Collection

Comprehensive real time test cell data collection enables applications to perform data analysis in real time. Below is a list of OneAPI data types and a brief description of information that can be collected for each data type.

Table Error! No text of specified style in document.-2. OneAPI Data Types

Data Type	Information Collected
PRODUCTION_LOTSTART	Lot Start event includes the following information: • Lot start time • Lot ID, Sublot ID • Test type • Site list • Prober/Handler, LB ...
PRODUCTION_LOTEND	Lot End event includes the following information: • Lot complete time • Lot ID ...
PRODUCTION_WAFERSTART	Wafer Start event includes the following information: • Wafer start time • Wafer ID • Wafer layout information ...
PRODUCTION_WAFEREND	Wafer End event includes the following information: • Wafer complete time • Wafer ID ...
PRODUCTION_TESTSTART	Test Start event includes the following information: • Test start time • Site list
PRODUCTION_TESTEND	Test End event includes the following information: • Test complete time • Bin result of each site • Part ID of each site • X/Y Coordinates of each site • Test time ...
PRODUCTION_TESTFLOWSTART	Test Flow Start event includes the following information: • Test flow start time • Test flow name
PRODUCTION_TESTFLOWEND	Test Flow End event includes the following information: • Test flow end time • Test flow name

PRODUCTION_TESTSUITESTART	Test Suite Start event includes the following information: • Test Suite start time • Test Suite name
PRODUCTION_TESTSUITEEND	Test Suite End event includes the following information: • Test Suite end time • Test Suite name • Release Tester time
MEASURED_PARAMETRIC	Parametric Test Result event includes: • Test method information: number, name, lo/hi limit, unit • Result of each site: pass/fail, measured value, etc. ...
MEASURED_FUNCTIONAL	Functional Test Result event includes: • Test method information: number, name • Result of each site: pass/fail, cycle count, fail-pins etc.
MEASURED_MULTI_PARAM	Multi-Parametric Test Result event includes: • Test method information: number, name, lo/hi limit, unit • Multiple measured values of each site ...
MEASURED_SCAN	Scan Test Result event includes: • Test method info: number • Result of each Site: total cycle count, fail cycle count • Result of each Pattern: name, pins information • Result of each Pin: name, failing cycles ...
DEVICE	Device data includes: • Test Program info: name, path, pin config file etc. • Bin table: soft-bin, hard-bin • Pin data: index, type, name, number etc. • Pattern data: index, name, pattern files etc.
USERDEFINED	User Defined data includes: • User specified variable name and its value
DATALOGTEXT	Datalog Text data includes: • Datalog text time stamp • Datalog text expression

Tester control instruction

ActionManager will help user to define instructions which need host controller site to execute.

control type	description
settest	Define which test suites should be bypassed or not
setpat	Define which parallel group bypassed or not
setprogvar	Define the value of test program value should be changed to
setrestore	Define restoring all change
set_wait	Define wait time and and reason
set_pause	Define production pause
set_message	Define message to show in production
get	Get action instructions
clean	Clean all actions in memory

1.1.3 Python API

This is a set of static functions which are used for communication between the user application and ACS Nexus. During the lifecycle of the application, no instance of this class needs to be generated.

1.1.3.1 class Interface

```
def connect(me: AppInfo, cmdPort: int = 0, dataPort: int = 0) -> int
```

Description	Initiate the request to connect to ACS Nexus. Normally, the application only needs to input AppInfo. Connection must be established before sending control commands or receiving data.
Parameter	Input: • Information of the application • Whether to enable Nexus Data Streaming and Control • Whether to enable TPService for communication with NexusTPI
Return	0 – initial connection succeeded -1 – failed to enable Nexus Data Streaming and Control -2 – failed to enable TPService -3 – unknown location -4 – not support NOTES: The return value 0 does not mean a connection with ACS Nexus has been established. Another interface getConnectionState() can provide the current connection state of the command channel. For a return value of -1, a possible reason could be failed to connect to nexus-broker.

	For a return value of -2, a possible reason could be TPService port is occupied. (the default TPService port is 21122). For a return value of -3, a possible reason could be OneAPI runs in an unknown location. For a return value -4, a possible reason could be OneAPI does not support the corresponding feature in the current location. For any exception case, all enabled connection will be disconnected.
--	--

```
def disconnect() -> int
```

Description	Disconnects from ACS Nexus.
Return	0 disconnect succeeded -1 disconnect failed

```
def registerMonitor(myMonitor: Monitor) -> None
```

Description	Registers the event monitor to receive real-time data through a callback function. For additional details, refer to class Monitor .
Return	0 – disconnect success -1 – disconnect fail

```
def getConnectionState(cmdChannel: int) -> None
```

Description	Get the connection status of the command channel. For command channel, the status value is 0 only when the current status is established, and communication is available. Otherwise, the status value is – 1. The application should determine whether the related API can be executed according to the connection state. For example, when sending a control command, if the status of the command channel is not 0, the command will not be sent to ACS Nexus.
Parameter	Output: The status of command channel (for sending command to ACS Nexus).

```
def sendCommand(tc: TestCell, cmd: Command ) -> int
```

Description	Send the command to the connected ACS Nexus. The return value of this interface indicates whether the action of sending command is successful. ACS Nexus will execute the command asynchronously after receiving the command.
Parameter	Input: TestCell information (refer to class TestCell)

	• Command information (refer to class Command)
Return	0 – command successfully sent to ACS Nexus -1 – failed to send command to ACS Nexus -3 – unknown location -4 – not support

1.1.3.2 class Monitor

```
def consumeData(tc: TestCell, data: NexusData) -> None
```

Description	This is a pure virtual function that needs to be inherited and overridden. As a callback function, it will be triggered every time ACS Nexus data arrives. NOTE1: When the function is triggered, the thread will be blocked. It cannot be triggered again until the callback function completes its processing logic and returns. Therefore, attention should be given to the efficiency of processing or consideration given to implementing asynchronous processing in the code. NOTE 2: You must call the corresponding <i>get</i> and <i>query</i> interfaces inside this function and save the values, otherwise the data will be cleared.
Parameter	• TestCell information (refer to class TestCell) • ACS Nexus data (refer to Class class NexusData)

```
def consumeTPSend( tc: TestCell, data: str) -> None
```

Description	This is a function that needs to be inherited and overridden. As a callback function, it will be triggered every time TP sends data via NexusTPI. NOTE: When the function is triggered, the thread will be blocked. It cannot be triggered again until the callback function completes its processing logic and returns. Therefore, attention should be given to the efficiency of processing or consideration given to implementing asynchronous processing in the code.
Parameter	• TestCell information (refer to class TestCell) • Data type is string

```
def consumeTPRequest ( tc: TestCell, request: str) -> str
```

Description	This is a function that needs to be inherited and overridden. As a callback function, it will be triggered every time TP initiates a request via NexusTPI. NOTE: When the function is triggered, the thread will be blocked. It cannot be triggered again until the callback function completes its processing logic and returns. Therefore, attention should be given to the efficiency of processing or consideration given to implementing asynchronous processing in the code.
Parameter	• TestCell information (refer to class TestCell) • Request type is string in JSON format
Return	string: after handle request, return your response to NexusTPI.

1.1.3.3 enum DataType

Description	Indicate the type of ACS Nexus data. • DATA_TYP_PRODUCTION_LOTSTART • DATA_TYP_PRODUCTION_WAFERSTART • DATA_TYP_PRODUCTION_TESTSTART • DATA_TYP_PRODUCTION_TESTEND • DATA_TYP_PRODUCTION_WAFEREND • DATA_TYP_PRODUCTION_LOTEND • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_DEVICE • DATA_TYP_USERDEFINED • DATA_TYP_FILE • DATA_TYP_PRODUCTION_TESTFLOWSTART • DATA_TYP_PRODUCTION_TESTFLOWEND • DATA_TYP_DATALOGTEXT • DATA_TYP_IDENTIFICATION • DATA_TYP_DEVICE_PIN • DATA_TYP_MEASURED_SCAN • DATA_TYP_DEVICE_PATTERN • DATA_TYP_MEASUREMENT • DATA_TYP_PRODUCTION_TESTSUITESTART • DATA_TYP_PRODUCTION_TESTSUITEEND
--------------------	---

1.1.3.4 class NexusData

Each time the callback function `consumeData` is triggered, a reference to the `NexusData` object will be passed in as a parameter. Detailed values can be obtained by calling the member functions of this class.

NOTE: The corresponding member functions must be called according to the type of data to obtain the current valid value.

[Production Event] Start of Lot Data Type DATA_TYP_PRODUCTION_LOTSTART Occurs: Once at the beginning of the first touchdown per LOT Contains: All the global information for this LOT	
Member Function	Return Value(s) Description
<code>get_Timezone() -> int32</code>	The time zone where ACS Nexus is running
<code>get_SetupTime() -> uint64</code>	Date and time when the test program was started (in microseconds)
<code>get_TimeStamp() -> uint64</code>	Date and time when the first part was tested (in microseconds)
<code>get_StationNumber() -> int32</code>	Tester station number
<code>get_ModeCode() -> str</code>	Test mode code
<code>get_RetestCode() -> str</code>	Lot retest code
<code>get_ProtectionCode() -> str</code>	Data protection code
<code>get_BurnTimeMinutes() -> int32</code>	Burn-in time (in minutes)
<code>get_CommandCode() -> str</code>	Command mode code of the tester
<code>get_LotId() -> str</code>	Lot ID
<code>get_PartType() -> str</code>	Part type or product ID
<code>get_NodeName() -> str</code>	Hostname of the tester system controller
<code>get_TesterType() -> str</code>	Tester type
<code>get_JobName() -> str</code>	Test program name
<code>get_JobRevision() -> str</code>	Test program revision number
<code>get_SublotId() -> str</code>	Sublot ID
<code>get_OperatorName() -> str</code>	Operator name or ID at setup time
<code>get_TesterosType() -> str</code>	Tester software type
<code>get_TesterosVersion() -> str</code>	Tester software version number
<code>get_TestType() -> str</code>	Type of Lot (PACKAGE_TEST / WAFER_TEST)
<code>get_TestStepCode() -> str</code>	Test phase or step code
<code>get_TestTemperature() -> str</code>	Test temperature
<code>get_UserText() -> str</code>	User-defined text
<code>get_AuxiliaryFile() -> str</code>	Name of auxiliary data file
<code>get_PackageType() -> str</code>	Package type
<code>get_FamilyId() -> str</code>	Product family ID
<code>get_DateCode() -> str</code>	Date code
<code>get_FacilityId() -> str</code>	Test facility ID
<code>get_FloorId() -> str</code>	Test floor ID
<code>get_ProcessId() -> str</code>	Fabrication process ID

get_OperationFreq() -> str	Operation frequency or step
get_SpecName() -> str	Test specification name
get_SpecVersion() -> str	Test specification version number
get_FlowId() -> str	Testflow ID
get_SetupId() -> str	Test setup ID
get_DesignRevision() -> str	Device design revision
get_EngineeringLotId() -> str	Engineering lot ID
get_RomCode() -> str	ROM code ID
get_SerialNumber() -> str	Tester serial number
get_SupervisorName() -> str	Supervisor name or ID
get_HeadNumber() -> uint32	Test head number
get_SiteGroupNumber() -> uint32	Site group number (station number)
get_SiteCount() -> uint32	Number of active sites described in this record
get_TotalHeadSiteList() -> list	Array of test site numbers with head information
get_ProberHandlerType() -> str	Handler or prober type
get_ProberHandlerId() -> str	Handler or prober ID
get_ProbecardType() -> str	Probe card type
get_ProbecardId() -> str	Probe card ID
get_LoadboardType() -> str	DUT board type
get_LoadboardId() -> str	DUT board ID
get_DibType() -> str	DIB board type
get_DibId() -> str	DIB board ID
get_CableType() -> str	Interface cable type
get_CableId() -> str	Interface cable ID
get_ContactorType() -> str	Handler contactor type
get_ContactorId() -> str	Handler contactor ID
get_LaserType() -> str	Laser type
get_LaserId() -> str	Laser ID
get_ExtraEquipType() -> str	Extra equipment type
get_ExtraEquipId() -> str	Extra equipment ID

[Production Event] End of Lot DataType DATA_TYP_PRODUCTION_LOTEND Occurs: Once at the ending of the last touchdown per LOT Contains: Information that compliments the Lot	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time last part of the lot was tested (in microseconds)
get_DisPositionCode() -> str	Lot disposition code
get_UserDescription() -> str	Lot description supplied by the user
get_ExecDescription() -> str	Lot description supplied by the exec

[Production Event] Start of Wafer DataType DATA_TYP_PRODUCTION_WAFERSTART Occurs: Once at the beginning of the first touchdown per WAFER Contains: All the configuration information for the tested wafer	
Member Function	Return Value Description
get_WaferSize() -> float	Diameter of wafer in WF_UNITS
get_DieHeight() -> float	Height of die in WF_UNITS
get_DieWidth() -> float	Width of die in WF_UNITS
get_WaferUnits() -> uint32	Unit for wafer and die dimensions
get_WaferFlat() -> str	Orientation of wafer flat
get_CenterX() -> int32	X-coordinate of center die on wafer
get_CenterY() -> int32	Y-coordinate of center die on wafer
get_PositiveX() -> str	Positive X-direction of wafer
get_PositiveY() -> str	Positive Y-direction of wafer
get_HeadNumber() -> uint32	Test head number
get_SiteGroupNumber() -> uint32	Site group number
get_TimeStamp() -> uint64	Date and time when the first part of wafer was tested (in microseconds)
get_WaferId() -> str	Wafer ID

[Production Event] End of Wafer DataType DATA_TYP_PRODUCTION_WAFEREND Occurs: Once at the ending of the last touchdown per WAFER Contains: The result information for the tested wafer	
Member Function	Return Value Description
get_HeadNumber() -> uint32	Test head number
get_SiteGroupNumber() -> uint32	Site group number
get_TimeStamp() -> uint64	Date and time when the last part of wafer was tested (in microseconds)
get_TestCount() -> uint32	Number of tested parts
get_RetestCount() -> uint32	Number of retested parts
get_GoodCount() -> uint32	Number of tested parts that have passed
get_WaferId() -> str	Wafer ID
get_UserDescription() -> str	Wafer description supplied by user
get_ExecDescription() -> str	Wafer description supplied by exec

[Production Event] End of Test DataType DATA_TYP_PRODUCTION_TESTEND Occurs: Once at the ending of each touchdown Contains: Site, part, and result information for this touchdown	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time when the part completes test (in microseconds)
get_ResultCount() -> uint32	Count of test result
query_HeadSite(uint32 index) -> uint32	Test site number with head information
query_PartFlag(uint32 index) -> uint32	Part information flags
query_SBinResult(uint32 index) -> uint32	Software bin
query_HBinResult(uint32 index) -> uint32	Hardware bin
query_XCoord(uint32 index) -> int32	Wafer X-coordinate
query_YCoord(uint32 index) -> int32	Wafer Y-coordinate
query_TestTime(uint32 index) -> uint32	Elapsed test time in microseconds
query_PartId(uint32 index) -> str	Part identification
query_PartText(uint32 index) -> str	Part description text

[Production Event] Start of Test DataType DATA_TYP_PRODUCTION_TESTSTART Occurs: Once at the beginning of each touchdown Contains: Site information for this touchdown	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time when the part starts to test (in microseconds)
get_HeadSiteList() -> list	Array of test site number with head information
query_HeadSite(index: uint32) -> uint32	Test site number with head information
query_XCoord(index: uint32) -> int32	Wafer X-coordinate
query_YCoord(index: uint32) -> int32	Wafer Y-coordinate

[Production Event] Start of Test Flow DataType DATA_TYP_PRODUCTION_TESTFLOWSTART Occurs: Once at the beginning of each test flow Contains: Test flow name	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time when the test flow starts (in microseconds)
get_TestFlowName() -> str	Test flow name

[Production Event] End of Test Flow DataType DATA_TYP_PRODUCTION_TESTFLOWEND Occurs: Once at the ending of each test flow Contains: Test flow name	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time when the test flow completes (in microseconds)
get_TestFlowName() -> str	Test flow name

[Production Event] Start of Test Suite DataType DATA_TYP_PRODUCTION_TESTSUITESTART Occurs: Once at the beginning of each test suite Contains: Test suite name	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time when the test suite starts (in microseconds)
get_TestSuite() -> str	Test suite name

[Production Event] End of Test Suite DataType DATA_TYP_PRODUCTION_TESTSUITEEND Occurs: Once at the ending of each test suite Contains: Test suite name, and release-tester time	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time when the test suite ends (in microseconds)
get_TestSuite() -> str	Test suite name
get_ReleaseTesterTimeStamp() -> uint64	Date and time when the foreground time ends (in microseconds)

[Measured Value] Parametric Test Result DataType DATA_TYP_MEASURED_PARAMETRIC Occurs: Once per parametric test item during the test of each touchdown Contains: Results information for all tested sites, and basic information of the test item	
Member Function	Return Value Description
get_ResultCount() -> uint32	Count of result for the test item
query_HeadSite(uint32 index) -> uint32	Test site number with head information
query_TestNumber(uint32 index) -> uint32	Test number
query_TestText(uint32 index) -> str	Test description text or label ¹
query_LowLimit(uint32 index) -> float	Low test limit value
query_HighLimit(uint32 index) -> float	High test limit value
query_Unit(uint32 index) -> str	Test units
query_TestFlag(uint32 index) -> str	Test flags (fail, alarm, etc.)
query_Result(uint32 index) -> float	Test result
query_TestSuite(uint32 index) -> str	Test suite name
query_MeasurementName(index: uint32) -> str	The unique fully qualified name of measurement

¹ In SmarTest 7, the priority is the test comment, followed by "Test suite name: test name." If there is a pin name, it will be "Test suite name: test name : pin name." In SmarTest 8, it will be always the test descriptor.

[Measured Value] Functional Test Result DataType DATA_TYP_MEASURED_FUNCTIONAL Occurs: Once per functional test item during the test of each touchdown Contains: Results information for all tested sites, and basic information of the test item	
Member Function	Return Value Description
get_ResultCount() -> uint32	Count of result for the test item
query_HeadSite(uint32 index) -> uint32	Test site number with head information
query_TestNumber(uint32 index) -> uint32	Test number
query_TestText(uint32 index) -> str	Test description text or label
query_TestFlag(uint32 index) -> str	Test flags (fail, alarm, etc.)

query_CycleCount(uint32 index) -> uint32	Vector cycle count
query_NumberFail(uint32 index) -> uint32	The number of logic pin names with one or more failures
query_PinResults(index: uint64) -> list	List of fail Pin index which belong to the given Result index
query_PinName(pinID: uint64) -> str	Pin name
query_FailCycles(pinID: uint64) -> list	List of fail cycle numbers which belong to the given Pin
query_VectNam(index: uint32) -> str	Vector module pattern name
query_TestSuite(uint32 index) -> str	Test suite name
query_MeasurementName(index: uint32) -> str	The unique fully qualified name of measurement
query_PinIndexbyID(pinID: uint64) -> uint32	Pin index

[Measured Value] Multiple-Result Parametric Test Record DataType DATA_TYP_MEASURED_MULTI_PARAM Occurs: Once per multiple-result parametric test item during the test of each touchdown Contains: Results information for all tested sites, and basic information of the test item	
Member Function	Return Value Description
get_ResultCount() -> uint32	Count of result for the test item
query_HeadSite(uint32 index) -> uint32	Test site number with head information
query_TestNumber(uint32 index) -> uint32	Test number
query_TestText(uint32 index) -> str	Test description text or label
query_LowLimit(uint32 index) -> float	Low test limit value
query_HighLimit(uint32 index) -> float	High test limit value
query_Unit(uint32 index) -> str	Test units
query_TestFlag(uint32 index) -> str	Test flags (fail, alarm, etc.)
query_Results(uint32 index) -> list	List of Test results which belong to the given Result index
query_PinResults(index: uint64) -> list	List of Pin index which belong to the given Result index
query_PinName(pinID: uint64) -> str	Pin name
query_TestSuite(uint32 index) -> str	Test suite name
query_MeasurementName(index: uint32) -> str	The unique fully qualified name of measurement
query_PinIndexbyID(pinID: uint64) -> uint32	Pin index

[Notify] Scan Test Result DataType DATA_TYP_MEASURED_SCAN Occurs: Once per scan test item during the test of each touchdown Contains: Results information for all tested sites, and basic information of the test item	
Member Function	Return Value Description
get_ResultCount() -> uint32	Count of result for the test item
query_HeadSite(index: uint32) -> uint32	Test site number with head information
query_TestNumber(index: uint32) -> uint32	Test number
query_TestText(index: uint32) -> str	Test description text or label
query_TestFlag(index: uint32) -> str	Test flags (fail, alarm, etc.)
query_TotalCycleCount(index: uint32) -> int32	Total number of cycles
query_FailCycleCount(index: uint32) -> int32	Total number of failing cycles
query_OpSequence(index: uint32) -> str	Operating sequence
query_PatternResults(index: uint32) -> list	List of pattern index which belong to the given Result
query_PatternName(patternID: uint64) -> str	Pattern name
query_PinResults(index: uint64) -> list	List of pin index which belong to the given Pattern
query_PinName(pinID: uint64) -> str	Pin name
query_FailCycles(pinID: uint64) -> list	List of fail cycle numbers which belong to the given Pin
query_TestSuite(uint32 index) -> str	Test suite name
query_MeasurementName(index: uint32) -> str	The unique fully qualified name of measurement
query_PinIndexbyID(pinID: uint64) -> uint32	Pin index

[Device Data] DataType DATA_TYP_DEVICE Occurs: Once for each lot, when all the required data collection is completed Contains: Device information	
Member Function	Return Value Description
get_TestProgramDir() -> str	The absolute path of the folder of the active test program
get_TestProgramName() -> str	The fully qualified name of the active test program
get_TestProgramPath() -> str	The absolute path of the active test program file
get_PinConfig() -> str	The fully qualified name of the active DUT board description file
get_ChannelAttribute() -> str	The fully qualified name of the active Channel attributes file
get_BinInfoCount() -> uint32	Count of defined soft bin of the active test program
query_SBinNumber(uint32 index) -> uint32	The number of the soft bin
query_SBinName(uint32 index) -> str	The description of the soft bin
query_SBinType(uint32 index) -> uint32	The type of the soft bin
query_HBinNumber(uint32 index) -> uint32	The corresponding hard bin number
query_HBinName(uint32 index) -> str	The description of the corresponding hard bin
query_HBinType(uint32 index) -> uint32	The type of the corresponding hard bin

[Notify] Pin Data DataType DATA_TYP_DEVICE_PIN Occurs: Once for each lot, when all the required data collection is completed. Contains: Pin information	
Member Function	Return Value Description
get_PinInfoCount() -> uint32	The count of pin information
query_PinIndex(index: uint32) -> uint32	Pin index
query_ChannelType(index: uint32) -> uint32	Channel type
query_ChannelName(index: uint32) -> str	Channel name
query_PhysicalName(index: uint32) -> str	Name of physical pin
query_LogicalName(index: uint32) -> str	Name of logical pin
query_HeadNumber(index: uint32) -> str	Test head number
query_SiteNumber(index: uint32) -> uint32	Test site number

[Notify] Pattern Data DataType DATA_TYP_PATTERN Occurs: Whenever a pattern variable is collected. Contains: Information on a pattern profile for a specific scan test.	
Member Function	Return Value Description
get_PatternCount() -> uint32	The count of patterns
query_OpSequence(index: uint32) -> str	Operating sequence
query_PatternLabels(index: uint32) -> list	List of pattern labels

[User Defined Data] DataType DATA_TYP_USERDEFINED Occurs: Whenever the user-defined variable is collected Contains: Variable and its value information	
Member Function	Return Value Description
get_UserDefined() -> dict	Map of user defined data <variable, value>

[Notify] File Transfer DataType DATA_TYP_FILE Occurs: When each file transfer is completed Contains: The file name	
Member Function	Return Value Description
get_FileName() -> str	The absolute path of the received file

[Notify] Datalog Text oneapi::DataType DATA_TYP_DATALOGTEXT Occurs: Whenever a datalog-text variable is collected Contains: The expression of datalog-text	
Member Function	Return Value Description
get_TimeStamp() -> uint64	Date and time when the test flow completes (in microseconds)
get_DataLogText() -> str	The expression of datalog text

[Notify] Measurement Data oneapi::DataType DATA_TYP_MEASUREMENT Occurs: Once a measurement has ended Contains: The information of measurement	
Member Function	Return Value Description
get_MeasurementName() -> str	The unique fully qualified name of measurement
get_GroupCount() -> uint32	Total count of parallel groups
query_GroupName(index: uint32) -> str	The name of the parallel group
query_GroupBypassed(index: uint32) -> int32	Whether the parallel group was bypassed
query_GroupSites(index: uint32) -> list	The number of the site(s) where the parallel group was bypassed
get_SequenceCount() -> uint32	Total count of the operating sequence
query_SequenceName(index: uint32) -> str	The name of the operating sequence
query_SequenceBypassed(index: uint32) -> int32	Whether the operating sequence was bypassed
query_SequenceSites(index: uint32) -> list	The number of site where the operating sequence was bypassed
query_SequenceGroupResults(index: uint32) -> list	list of unique "GroupID" for the specific sequence
query_SequenceGroupName(groupID: uint32) -> str	The name of the parallel group that belongs to current sequence
query_SequenceGroupBypassed(groupID: uint32) -> int32	Whether the parallel group was bypassed
query_SequenceGroupSites(groupID: uint32) -> list	The number of the site where the parallel group was bypassed

1.1.3.4.1 Function Introduction

getType() -> DataType

Description	Get the type of data currently consumed. NOTE: This function must be called before all other functions in the consumeData code.
Return	One of the following: • DATA_TYP_PRODUCTION_LOTSTART • DATA_TYP_PRODUCTION_WAFERSTART • DATA_TYP_PRODUCTION_TESTSTART • DATA_TYP_PRODUCTION_TESTEND • DATA_TYP_PRODUCTION_WAFEREND • DATA_TYP_PRODUCTION_LOTEND • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_DEVICE • DATA_TYP_USERDEFINED • DATA_TYP_FILE • DATA_TYP_PRODUCTION_TESTFLOWSTART • DATA_TYP_PRODUCTION_TESTFLOWEND • DATA_TYP_DATALOGTEXT • DATA_TYP_IDENTIFICATION • DATA_TYP_DEVICE_PIN • DATA_TYP_MEASURED_SCAN • DATA_TYP_DEVICE_PATTERN • DATA_TYP_MEASUREMENT • DATA_TYP_PRODUCTION_TESTSUITESTART • DATA_TYP_PRODUCTION_TESTSUITEEND

get_TimeStamp() -> uint64

Description	Get the date and time in microseconds. Valid for: • DATA_TYP_PRODUCTION_LOTSTART • DATA_TYP_PRODUCTION_LOTEND • DATA_TYP_PRODUCTION_WAFERSTART • DATA_TYP_PRODUCTION_WAFEREND • DATA_TYP_PRODUCTION_TESTSTART • DATA_TYP_PRODUCTION_TESTEND • DATA_TYP_PRODUCTION_TESTFLOWSTART • DATA_TYP_PRODUCTION_TESTFLOWEND • DATA_TYP_PRODUCTION_TESTSUITESTART • DATA_TYP_PRODUCTION_TESTSUITEEND
Return	If by one of the above events, returns a valid value. For example, if event is DATA_TYP_PRODUCTION_LOTSTART, then the value represents the timestamp of Lot_Start. Otherwise, returns 0.

get_TestProgramDir() -> str

Description	Get the absolute path of the folder of the active test program. Valid always, refreshed by DATA_TYP_DEVICE.
Return	Valid value.

get_TestProgramName() -> str

Description	Get the fully qualified name of the active test program. Valid always, refreshed by DATA_TYP_DEVICE.
Return	Valid value.

get_TestProgramPath() -> str

Description	Get the absolute path of the active test program file. Valid always, refreshed by DATA_TYP_DEVICE.
Return	Valid value.

get_PinConfig() -> str

Description	Get the fully qualified name of the active DUT board description file. Valid always, refreshed by DATA_TYP_DEVICE.
Return	Valid value.

get_ChannelAttribute() -> str

Description	Get the fully qualified name of the active Channel attributes file. Valid always, refreshed by DATA_TYP_DEVICE.
Return	Valid value.

get_BinInfoCount() -> uint32

Description	Get the count of defined soft bin of the active test program. Valid always, refreshed by DATA_TYP_DEVICE.
Return	Valid value.

query_SBinNumber(index: uint32) -> uint32

Description	Query the number of the soft bin. Valid always, refreshed by DATA_TYP_DEVICE.
Parameter	Input: Index number
Return	Returns a valid value if valid input index. Returns 65535 if input index is out of bounds.

query_SBinName(index: uint32) -> str

Description	Query the description of the soft bin. Valid always, refreshed by DATA_TYP_DEVICE.
Parameter	Input: Index number
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

query_SBinType(index: uint32) -> uint32

Description	Query the type of soft bin. Valid always, refreshed by DATA_TYP_DEVICE.
Parameter	Input: Index number
Return	If valid input index, returns: 0 – PASS 1 – FAIL 2 – UNKONWN If input index is out of bounds, returns 2.

query_HBinNumber(index: uint32) -> uint32

Description	Query the number of hard bin. Valid always, refreshed by DATA_TYP_DEVICE.
Parameter	Input: Index number
Return	Returns a valid value if valid input index. Returns 65535 if input index is out of bounds.

query_HBinName(index: uint32) -> str

Description	Query the description of the hard bin. Valid always, refreshed by DATA_TYP_DEVICE.
Parameter	Input: Index number
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

query_HBinType(index: uint32) -> uint32

Description	Query the type of hard bin. Valid always, refreshed by DATA_TYP_DEVICE.
Parameter	Input: Index number
Return	If valid input index, returns: 0 – PASS 1 – FAIL 2 – UNKONWN If input index is out of bounds, returns 2.

get_PinInfoCount() -> uint32

Description	Get the total number of pin information. Valid for DATA_TYP_DEVICE_PIN.
Return	Returns a valid value if by above event. Otherwise, returns 0.

query_ChannelType(index: uint32) -> uint32

Description	Query the channel type. Valid for DATA_TYP_DEVICE_PIN.
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_ChannelName(index: uint32) -> str

Description	Query the channel name. Valid for DATA_TYP_DEVICE_PIN.
Return	Returns a valid value if valid input index. Otherwise, returns "" (empty string).

query_PhysicalName(index: uint32) -> str

Description	Query the name of the physical pin. Valid for DATA_TYP_DEVICE_PIN.
Return	Returns a valid value if valid input index. Otherwise, returns "" (empty string).

query_LogicalName(index: uint32) -> str

Description	Query the name of the logical pin. Valid for DATA_TYP_DEVICE_PIN.
Return	Returns a valid value if valid input index. Otherwise, returns "" (empty string).

query_HeadNumber(index: uint32) -> uint32

Description	Query the test head number. Valid for DATA_TYP_DEVICE_PIN.
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_SiteNumber(index: uint32) -> uint32

Description	Query the test site number. Valid for DATA_TYP_DEVICE_PIN.
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

get_PatternCount() -> uint32

Description	Get the count of patterns. Valid always, refreshed by DATA_TYP_DEVICE_PATTERN.
Return	Returns a valid value in all cases.

query_OpSequence(uint32 index) -> str

Description	Query the operating sequence. Valid always, refreshed by DATA_TYP_DEVICE_PATTERN & DATA_TYP_MEASURED_SCAN.
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

query_PatternLables(uint32 index) -> list

Description	Query the list of pattern labels. Valid for DATA_TYP_DEVICE_PATTERN.
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

get_UserDefined() -> dict

Description	Get the user defined key and value pairs. Valid for DATA_TYP_USERDEFINED.
Return	Returns valid value if by above event, otherwise returns map with no elements.

get_FileName() -> str

Description	Get the absolute path of the received file. The file is in the 'home' folder of the current user. Valid always, refreshed by DATA_TYP_FILE.
Return	Returns valid value if by above event, otherwise returns "" (empty string).

get_Timezone() -> int32

Description	Get the time zone where ACS Nexus is running (value in integer). Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns 0 in the case of: • Device Data • User Defined Data • File Transfer

get_SetupTime() -> uint64

Description	Get the date and time (in microseconds) when the test program was started. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns 0 in the case of: • Device Data • User Defined Data • File Transfer

get_StationNumber() -> uint32

Description	Get the tester station number (value in integer). Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns 0 in the case of: • Device Data • User Defined Data • File Transfer

get_ModeCode() -> str

Description	Get the test mode code. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_RetestCode() -> str

Description	Get the lot retest code. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ProtectionCode() -> str

Description	Get the data protection code. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_BurnTimeMinutes() -> uint32

Description	Get the burn-in time (in minutes). Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns 0 in the case of: • Device Data • User Defined Data • File Transfer

get_CommandCode() -> str

Description	Get the command mode code of the tester. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns " (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_LotId() -> str

Description	Get the lot ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_PartType() -> str

Description	Get the part type or product ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_NodeName() -> str

Description	Get the hostname of the tester system controller. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_TesterType() -> str

Description	Get the tester type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_JobName() -> str

Description	Get the test program name. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_JobRevision() -> str

Description	Get the test program revision number. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_SublotId() -> str

Description	Get the subplot ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_OperatorName() -> str

Description	Get the operator name or ID at setup time. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_TesterosType() -> str

Description	Get the software type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_TesterosVersion() -> str

Description	Get the tester software version number. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_TestType() -> str

Description	Get the type of lot. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value (PACKAGE_TEST or WAFER_TEST) in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_TestStepCode() -> str

Description	Get the test phase or step code. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_TestTemperature() -> str

Description	Get the test temperature. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_UserText() -> str

Description	Get the user defined text. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_AuxiliaryFile() -> str

Description	Get the name of the auxiliary data file. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_PackageType() -> str

Description	Get the package type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_FamilyId() -> str

Description	Get the product family ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_DateCode() -> str

Description	Get the date code. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_FacilityId() -> str

Description	Get the test facility ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_FloorId() -> str

Description	Get the test floor ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ProcessId() -> str

Description	Get the fabrication process ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_OperationFreq() -> str

Description	Get the operation frequency or step. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_SpecName() -> str

Description	Get the test specification name. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_SpecVersion() -> str

Description	Get the specification version number. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_FlowId() -> str

Description	Get the test flow ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_SetupId() -> str

Description	Get the test setup ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_DesignRevision() -> str

Description	Get the device design revision. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_EngineeringLotId() -> str

Description	Get the engineering lot ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_RomCode() -> str

Description	Get the ROM code ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_SerialNumber() -> str

Description	Get the tester serial number. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_SupervisorName() -> str

Description	Get the supervisor name or ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_HeadNumber() -> uint32

Description	Get the test head number. Valid by: • DATA_TYP_PRODUCTION_LOTSTART • DATA_TYP_PRODUCTION_WAFERSTART • DATA_TYP_PRODUCTION_WAFEREND
Return	In the case of Lot Start, returned value represents the test head number of the lot. In the case of Wafer Start or Wafer End, returned value represents test head number of the wafer. Otherwise, returns 0.

get_SiteGroupNumber() -> uint32

Description	Get the site group number. Valid by: • DATA_TYP_PRODUCTION_LOTSTART • DATA_TYP_PRODUCTION_WAFERSTART • DATA_TYP_PRODUCTION_WAFEREND
Return	In the case of Lot Start, returned value represents the site group number of the lot. In the case of Wafer Start or Wafer End, returned value represents site group number of the wafer. Otherwise, returns 0.

get_SiteCount() -> uint32

Description	Get the number of active sites described in this record. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns 0 in the case of: • Device Data • User Defined Data • File Transfer

get_TotalHeadSiteList() -> list

Description	Get all test site numbers. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns no element in the case of: • Device Data • User Defined Data • File Transfer

get_ProberHandlerType() -> str

Description	Get handler or prober type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ProberHandlerId() -> str

Description	Get the handler or prober ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ProbecardType() -> str

Description	Get the probe card type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ProbecardId() -> str

Description	Get the probe card ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_LoadboardType() -> str

Description	Get the DUT board type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_LoadboardId() -> str

Description	Get the DUT board ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_DibType() -> str

Description	Get the DIB board type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_DibId() -> str

Description	Get the DIB board ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_CableType() -> str

Description	Get the interface cable type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_CableId() -> str

Description	Get the interface cable ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ContactorType() -> str

Description	Get the handler contactor type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ContactorId() -> str

Description	Get the handler contactor ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_LaserType() -> str

Description	Get the laser type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_LaserId() -> str

Description	Get the laser ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ExtraEquipType() -> str

Description	Get extra equipment type. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_ExtraEquipId() -> str

Description	Get extra equipment ID. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of: • Production Events • Measured Value Events Returns "" (empty string) in the case of: • Device Data • User Defined Data • File Transfer

get_DisPositionCode() -> str

Description	Get lot disposition code. Refreshed by DATA_TYP_PRODUCTION_LOTSTART.
Return	Returns a valid value in the case of the above event. Otherwise, returns "" (empty string)

get_UserDescription() -> str

Description	Get the description supplied by the user. Valid by: • DATA_TYP_PRODUCTION_LOTEND • DATA_TYP_PRODUCTION_WAFEREND
Return	In the case of Lot End, value represents the lot description supplied by the user. In the case of Wafer End, value represents the wafer description supplied by the user. Otherwise, returns "" (empty string).

get_ExecDescription() -> str

Description	Get the description supplied by the executive. Valid by: • DATA_TYP_PRODUCTION_LOTEND • DATA_TYP_PRODUCTION_WAFEREND
Return	In the case of Lot End, value represents the lot description supplied by the user. In the case of Wafer End, value represents the wafer description supplied by the user. Otherwise, returns "" (empty string).

get_WaferSize() -> float

Description	Get the diameter of the wafer in WF_UNITS. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns 0.0 in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_DieHeight() -> float

Description	Get the height of the die in WF_UNITS. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns 0.0 in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_DieWidth() -> float

Description	Get the width of the die in WF_UNITS. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns 0.0 in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_WaferUnits() -> uint32

Description	Get the unit for wafer and die dimensions. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns 0 in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_WaferFlat() -> str

Description	Get the orientation of the wafer flat. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns "" (empty string) in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_CenterY() -> int32

Description	Get the Y-coordinate of the center die on the wafer. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns -32787 in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_CenterX() -> int32

Description	Get the X-coordinate of the center die on the wafer. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns -32787 in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_PositiveX() -> str

Description	Get the positive X-direction of the wafer. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns "" (empty string) in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_PositiveY() -> str

Description	Get the positive Y-direction of the wafer. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns "" (empty string) in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_WaferId() -> str

Description	Get the wafer ID. Refreshed by: • DATA_TYP_PRODUCTION_WAFERSTART
Return	Returns a valid value in the case of: • Production Events (except for Lot Start) • Measured Value Events Returns "" (empty string) in the case of: • Lot Start • Device Data • User Defined Data • File Transfer

get_TestedListCount() -> uint32

Description	Get the number of the tested parts. Refreshed by: • DATA_TYP_PRODUCTION_WAFEREND
Return	Returns a valid value in the case above, otherwise returns 0.

get_RetestedListCount() -> uint32

Description	Get the number of the retested parts. Refreshed by: • DATA_TYP_PRODUCTION_WAFEREND
Return	Returns a valid value in the case above, otherwise returns 0.

get_GoodCount() -> uint32

Description	Get the number of the tested parts that have passed. Refreshed by: • DATA_TYP_PRODUCTION_WAFEREND
Return	Returns a valid value in the case above, otherwise returns 0.

get_HeadSiteList() -> list

Description	Get array of test site number with head information. Valid for DATA_TYP_PRODUCTION_TESTSTART NOTE: “headsite” is a new concept proposed by OneAPI. It integrates head and site information into an integer. The following methods can convert headsite to head or site: • uint32 toHead(uint32 index); • uint32 toSite(uint32 index);
Return	Returns a valid value in the case above, otherwise returns no element.

get_ResultCount() -> uint32

Description	Get the count of the test result. Valid for: • DATA_TYP_PRODUCTION_TESTEND • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM
Return	In the case of the above events, returns a valid value. For example, if event is DATA_TYP_PRODUCTION_TESTEND, then the value represents the count of DUTs of the current touchdown. If event is DATA_TYP_MEASURED_FUNCTIONAL, then the value represents the count of functional test result for the current test item. Otherwise, returns 0

query_HeadSite(index: uint32) -> uint32;

Description	Query the test site number with head information. Valid by: • DATA_TYP_PRODUCTION_TESTEND • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_PRODUCTION_TESTSTART
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_PartFlag(index: uint32) -> str

Description	Query the part information flags. Valid by: DATA_TYP_PRODUCTION_TESTEND
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

query_SBinResult(index: uint32) -> uint32

Description	Query the soft bin number. Valid by: DATA_TYP_PRODUCTION_TESTEND
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_HBinResult(index: uint32) -> uint32

Description	Query the hard bin number. Valid by: DATA_TYP_PRODUCTION_TESTEND
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_XCoord(index: uint32) -> int32

Description	Query the X-coordinate. Valid by: - DATA_TYP_PRODUCTION_TESTEND - DATA_TYP_PRODUCTION_TESTSTART
Return	Returns a valid value if valid input index. Returns 65535 if input index is out of bounds.

query_YCoord(index: uint32) -> int32

Description	Query the Y-coordinate. Valid by: • DATA_TYP_PRODUCTION_TESTEND • DATA_TYP_PRODUCTION_TESTSTART
Return	Returns a valid value if valid input index. Returns 65535 if input index is out of bounds.

query_TestTime(index: uint32) -> uint32

Description	Query the elapsed test time in microseconds. Valid by: • DATA_TYP_PRODUCTION_TESTEND
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_PartId(index: uint32) -> str

Description	Query the part identification. Valid by: • DATA_TYP_PRODUCTION_TESTEND
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

query_PartText (index: uint32) -> str

Description	Query the part description text. Valid by: • DATA_TYP_PRODUCTION_TESTEND
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

query_TestNumber(index: uint32) -> uint32

Description	Get the number of current test item. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the number of the current parametric test. Returns "" (empty string) if input index is out of bounds.

query_TestText(index: uint32) -> str

Description	Query test description text or label. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the test text of the current parametric test. Returns "" (empty string) if input index is out of bounds.

query_LowLimit(index: uint32) -> float

Description	Query the low test limit value. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_MULTI_PARAM
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the low limit of the current parametric test. Returns 0.0 if input index is out of bounds.

query_HighLimit(index: uint32) -> float

Description	Query the high test limit value. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_MULTI_PARAM
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the high limit of the current parametric test. Returns 0.0 if input index is out of bounds.

query_Unit(index: uint32) -> str

Description	Query the test units. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_MULTI_PARAM
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the test unit of the current parametric test. Returns "" (empty string) if input index is out of bounds.

query_TestFlag(index: uint32) -> str

Description	Query the test flags, including fail, alarm, etc. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the test flags of the current parametric test. Returns "" (empty string) if input index is out of bounds.

NexusData::query_TestSuite(index: uint32) -> str

Description	Query the test suite name. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the test suite name of the current parametric test. Returns "" (empty string) if input index is out of bounds.

NexusData::query_MeasurementName(index: uint32) -> str

Description	Query the measurement name. Valid by: • DATA_TYP_MEASURED_PARAMETRIC • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. For example, if event is DATA_TYP_MEASURED_PARAMETRIC, then the value represents the measurement name of the current parametric test. Returns "" (empty string) if input index is out of bounds.

query_Result(index: uint32) -> float

Description	Query the test result. Valid by: • DATA_TYP_MEASURED_PARAMETRIC
Return	Returns a valid value if valid input index. Returns 0.0 if input index is out of bounds.

query_CycleCount(index: uint32) -> uint32

Description	Query the vector cycle count. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_NumberFail(index: uint32) -> uint32

Description	Query the vector cycle count. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_FailPins(index: uint32) -> list

Description	Query the list of failing pin bit field. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

query_VectNam(index: uint32) -> str

Description	Query the vector module pattern name. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

query_Results(index: uint32) -> list

Description	Query the list of all test results. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

get_TestFlowName() -> str

Description	Get the test flow name. Valid by: • DATA_TYP_PRODUCTION_TESTFLOWSTART • DATA_TYP_PRODUCTION_TESTFLOWEND
Return	Returns a valid value if valid input index. Returns "" (empty string) if input index is out of bounds.

get_DataLogText() -> str

Description	Get the data log text. Valid by: • DATA_TYP_DATALOGTEXT
Return	Returns a valid value in the case of the above event. Otherwise, returns "" (empty string).

query_TotalCycleCount(index: uint32) -> int32

Description	Query the total number of cycles. Valid by: • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_FailCycleCount(index: uint32) -> int32

Description	Query the total number of failing cycles. Valid by: • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Returns 0 if input index is out of bounds.

query_PatternResults(index: uint32) -> list

Description	Query the list of pattern index results. Valid by: • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

query_PatternName(index: uint32) -> str

Description	Query the pattern name. Valid by: • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Otherwise, returns "NOT EXIST".

query_PinResults(index: uint64) -> list

Description	Query the list of pin index results. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

query_PinName(pinID: uint64) -> str

Description	Query the pin name. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Otherwise, returns "NOT EXIST".

query_FailCycles(pinID: uint64) -> list

Description	Query the list of fail cycle numbers. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

get_PatternCount() -> uint32

Description	Get the total number of patterns. Valid by: • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if by above event. Otherwise, returns 0.

query_PatternLabels(index: uint32) -> list

Description	Query the list of pattern names. Valid by: • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

get_MeasurementName() -> str

Description	Get the measurement name. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if by above event. Otherwise, returns "" (empty string).

get_GroupCount() -> uint32

Description	Get the total count of parallel groups. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if by above event. Otherwise, returns 0.

query_GroupName(index: uint32) -> str

Description	Query the name of the parallel group. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if valid input index. Otherwise, returns "" (empty string).

query_GroupBypassed(index: uint32) -> int32;

Description	Query whether the parallel group was bypassed. Valid by: • DATA_TYP_MEASUREMENT
Return	If valid input index, returns one of the following: • 0: bypassed is set to 'false' • 1: bypassed is set to 'true' • -1: bypassed is not set Returns -1 if input index is out of bounds.

query_GroupSites(index: uint32) -> list

Description	Query the number of the site where the parallel group was bypassed. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if valid input index. Returns no element (meaning that all sites were bypassed) if input index is out of bounds.

get_SequenceCount() -> uint32

Description	Get the total count of operating sequence call. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if by above event. Otherwise, returns 0.

query_SequenceName(index: uint32) -> str

Description	Query the name of the operating sequence call. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if valid input index. Otherwise, returns "" (empty string).

query_SequenceBypassed(index: uint32) -> int32;

Description	Query whether the operating sequence call was bypassed. Valid by: • DATA_TYP_MEASUREMENT
Return	If valid input index, returns one of the following: • 0: bypassed is set to 'false' • 1: bypassed is set to 'true' • -1: bypassed is not set Returns -1 if input index is out of bounds.

query_SequenceSites(index: uint32) -> list

Description	Query the number of the site where the operating sequence call was bypassed. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if valid input index. Returns no element (meaning that all sites were bypassed) if input index is out of bounds.

query_SequenceGroupResults(index: uint32) -> list

Description	Query the list of unique "GroupID" for the specific sequence. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if valid input index. Returns no element if input index is out of bounds.

query_SequenceGroupName(GroupID: uint32) -> str

Description	Query the name of the parallel group for the current sequence. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if valid input index. Otherwise, returns "" (empty string).

query_SequenceGroupBypassed(groupID: uint32) -> int32;

Description	Query whether the parallel group was bypassed for the current sequence. Valid by: • DATA_TYP_MEASUREMENT
Return	If valid input index, returns one of the following: • 0: bypassed is set to 'false' • 1: bypassed is set to 'true' • -1: bypassed is not set Returns -1 if input index is out of bounds.

query_SequenceGroupSites(groupID: uint32) -> list

Description	Query the number of the site where the parallel group was bypassed for the current sequence. Valid by: • DATA_TYP_MEASUREMENT
Return	Returns a valid value if valid input index. Returns no element (meaning that all sites for the current sequence were bypassed) if input index is out of bounds.

get_TestSuite() -> str

Description	Query the name of the test suite. Valid by: • DATA_TYP_PRODUCTION_TESTSUITESTART • DATA_TYP_PRODUCTION_TESTSUITEEND
Return	Returns a valid value if by above events. Otherwise, returns "" (empty string).

get_ReleaseTesterTimeStamp() -> uint64

Description	Query the timestamp when the foreground time ends. Valid by: • DATA_TYP_PRODUCTION_TESTSUITEEND
Return	Returns a valid value if by above event. Otherwise, returns 0.

query_PinIndexbyID (pinID: uint64) -> uint32

Description	Query the pin index. Valid by: • DATA_TYP_MEASURED_FUNCTIONAL • DATA_TYP_MEASURED_MULTI_PARAM • DATA_TYP_MEASURED_SCAN
Return	Returns a valid value if valid pin id input. Otherwise, returns 0.

[ActionManager]			
Member Function	Parameter	Return Value	Timing to Call
settest	settest(testerid, suitelist, state) Ex: ActionManager.settest("testerA", "testA, testB", "0")	bool	Any time
setpat	setpat(testerid, testsuite, pattern, state) Ex: setpat("testerA", "testC", "patC", "1")	bool	Any time
setprogvar	setprogvar(testerid, tp_variable, site, value) Ex: setprogvar("smartest_host", "Var1", 1, "0");	bool	Any time
set_wait	set_wait(testerid, wait_time, reason) Ex: set_wait("smartest_host", 10, "temperature");	bool	Any time
set_message	set_message(testerid, reason) Ex: set_message("testerA", "Message Abcdee")	bool	Any time
get	get(testerid) Ex: get("smartest_host")	string	need to call at consumeTPRequest event
clean	clean(testerid) Ex: clean("smartest_host")	string	Any time

1.1.3.5 class AppInfo

This class describes the application information which is filled in by the developer and passed in as an argument when calling Interface.connect. It will be used as the identification of this OneAPI client in the interaction with ACS Nexus. For example, when ACS Nexus receives a control command, it will know who sent the command.

Member

name = ""

Description	The name of this application.
-------------	-------------------------------

vendor = ""

Description	The vendor of this application.
-------------	---------------------------------

version = ""

Description	The version of this application.
-------------	----------------------------------

1.1.4 Configuration

As a dynamic library, OneAPI will statically read the contents of the environment variable for initialization when the application process is started. The contents of the environment variable are described in the table below.

Table Error! No text of specified style in document.-3. OneAPI Environment Variable Descriptions

Item	Description
ONEAPI_DEBUG	Developers must include and call the interface provided by OneAPI in their application. OneAPI will record status and information when executing these commands. This environment variable is used to set the output mode of the records. Values: 0 -- no output 1 -- enable output INFO level logs to console 2 -- enable output INFO level logs to file 3 -- enable output INFO level logs to both console and file 4 -- enable output DEBUG level logs to console 5 -- enable output DEBUG level logs to file 6 -- enable output DEBUG level logs to both console and file NOTE: The path of local log file is fixed to <code>\${home}/.log/</code>. The name follows the rule indicated below: <code>ACS_ONEAPI_yyyy-mm-dd.log</code>. (for example, <code>ACS_ONEAPI_2023-02-21.log</code>) To write log files successfully, ensure that the running environment of the application meets one of the following conditions: • <code>\${home}/.log/</code> folder does not exist, and the application has permission to create it. <code>\${home}/.log/</code> folder exists, and the application has permission to write log file in it.

1.2 Supporting Containerized Application on the ACS Edge and ACS Unified Servers

ACS Nexus provides support for real-time data and control features to the containerized application running on the ACS Edge Server and ACS Unified Server, without requiring modification of the test program. Multiple containerized applications running on the ACS Edge and ACS Unified Server are supported.

The below diagram offers a general view of how ACS Nexus supports the containerized application running on the ACS Edge Server.

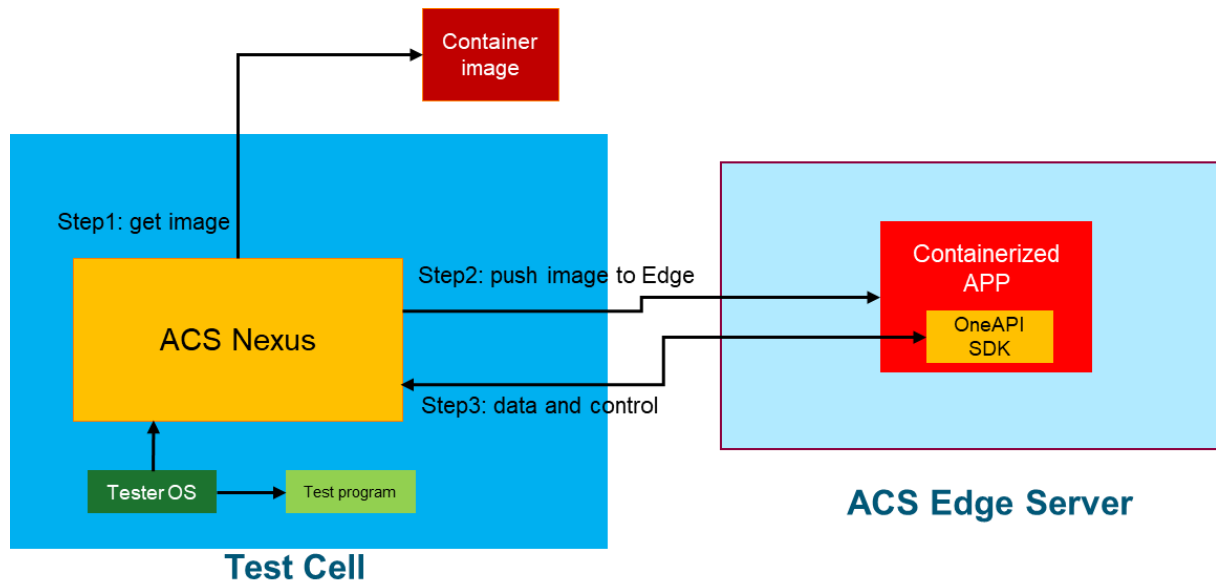


Figure Error! No text of specified style in document.-1. ACS Nexus Support for Containerized Application on ACS Edge Server

To support the containerized application running on the ACS Edge Server and ACS Unified, ACS Nexus must first be configured. ACS Nexus configuration is described in the subsections that follow.

Restarting ACS Nexus

For any changes to `acs_nexus.ini` to take effect, ACS Nexus should be stopped and manually restarted. Follow the procedure below to manually restart ACS Nexus.

1. Ensure SmarTest is already shutdown.
2. Open a command terminal and stop ACS Nexus by entering the following command (root user only):

```
#systemctl stop acs_nexus.service
```

3. To confirm that no ACS_Nexus processes are running, enter:

```
#ps -ef | grep acs/nexus | grep -v grep
```

The output for the above command should be empty.

4. Start ACS Nexus by entering the following command (root user only):

```
#systemctl start acs_nexus.service
```

5. To confirm that ACS_Nexus processes are started, enter:

```
#ps -ef | grep acs/nexus | grep -v grep
```

Below is an output example (SmarTest 7) for the above command.

```
root    112973      1  0 Jul07 ?        00:00:17 /opt/acs/nexus/bin/acs_nexus
root    112985      1  0 Jul07 ?        00:07:07 /opt/acs/nexus/bin/control_smt7 &
```

1.2.1 Auto Deploy Mode

NOTE: Auto Deploy mode is the default setting. In this mode, ACS Nexus supports production together with other RTDI products.

To dynamically deploy the specified Applications to the ACS Edge Server in real time according to production requirements, RTDI users must be able to query Application information from the ACS Container Hub through ACS Nexus before each production start. This mode is called Auto_Deploy mode, and the Application information file is called AppDescriptor.

To change the configuration to Auto Deploy mode, set Auto_Deploy.Enabled to 'true' in /opt/acs/nexus/conf/acs_nexus.ini.

When Auto_Deploy mode is enabled, ACS Nexus will use AppDescriptor (QueryAppDescriptor) as the configuration file to manage the Application lifecycle during production. ACS Nexus loads AppDescriptor when the SmarTest session starts. Therefore, make sure to call the query tool (QueryAppDescriptor) before the SmarTest session starts.

The path for AppDescriptor is /opt/acs/nexus/bin/QueryAppDescriptor. The table below provides information on the Command Line Interface parameters.

Table Error! No text of specified style in document.-4. Command Line Interface Parameter Details

Parameter	Meaning	Required	Description	Default Value
-h	Help	No	Show help information and then exit.	NA
-v	Version	No	Show the version and then exit.	NA
-c	Customer name	Yes	Set the customer name. Identifies the current customer so that it can be mapped to the according Container Hub Organization.	(empty)
-d	Device name	Yes	Set the device name.	(empty)
-f	Product family	No	Set the product family.	(empty)
-n	Test Program name	No	Set the test program name.	(empty)
-r	Test Program revision	No	Set test program revision.	(empty)
-u	URL	No	Set the basic url address for sending the query command. The default address is: https://unifiedserver.local/deployment/v1/application-descriptors/query	
-o	Output file	No	Set the absolute path of the output file, which stores the data part of the returned json format information. The default path is: /opt/acs/nexus/conf/app_descriptor.json	

1.2.2 Non-Auto Deploy Mode (Auto Deploy Mode Disabled)

NOTE: Non-Auto Deploy mode is used for engineering purposes only. To use Non-Auto Deploy mode, set `Auto_Deploy.Enabled` to `false` `/opt/acs/nexus/conf/acs_nexus.ini`.

The images configuration file (`images.json`) is used to define the containerized application that needs to be deployed and executed on the ACS Edge Server by ACS Nexus for a specific device.

The path for the `images.json` file is `/opt/acs/nexus/conf/images.json`. The contents and default values of this file are shown below. For a description of each item in this file, see [Table Error! No text of specified style in document.-5](#). For any modifications to this file to take effect, SmarTest must be restarted.

NOTE: Starting from ACS Nexus v2.0.0, the "images" file format is changed from `.ini` to `.json`. With this change, all image-related functions require ACS Edge Server version 2.3.1 or higher.

```
{
  "selector": {
    "device_name": "xxx.tp"
  },
  "edge": {
    "address": "xxx.xxx.xxx.xxx",
    "registry": {
      "address": "xxx.xxx.xxx.xxx",
      "user": "xxx",
      "password": "xxx"
    },
    "containers": [
      {
        "name": "mycontainer",
        "image": "test/myapp:latest",
        "requirements": {
          "gpu": false,
          "exposed_ports": [
            80,
            443
          ],
          "mapped_ports": [
            "9001:9001",
            "9002:9002/tcp"
          ]
        },
        "environment": {
          "VARNAME": "value",
          "VARNAME2": "value2"
        },
        "metrics": {
          "port": 9001,
          "path": "/metrics",
          "scrape_interval": 90,
          "scrape_timeout": 5
        },
        "volume_attachments": [
          "myvolume:rw",
          "myvolume2:ro"
        ]
      }
    ]
  },
}
```

```

    "volumes": [
      {
        "name": "myvolume",
        "image": "myorg-myproject/mydata:latest"
      },
      {
        "name": "myvolume2",
        "image": "myorg-myproject/mydata:latest"
      }
    ]
  }
}#####

```

Table Error! No text of specified style in document.-5. images.json Content Descriptions

Item	Description
selector	device_name Set the device name that will run.
edge	address Configure the Edge Server IP. Used for development testing only. Do not configure in production.
edge.registry	address The location of container registry, such as ACS Container Hub or Docker hub.
	user Authentication username for registry when pulling an image.
	password Authentication password for registry when pulling an image.
edge.containers	name The name of the container.
	image The name of the image.
	environment Set the environment variable for the container.
	metrics Set the metrics for the container.
edge.containers.requirements	gpu Configure whether the corresponding container enables GPU. Valid value: true/false
	exposed_ports This option can be used to expose container ports.
	mapped_ports Use this option to expose and publish container ports.

edge.containers.volume_attachments	This option is used to set the attach mounted volumes.
edge.containers.volumes	name Define the volume name to create.
	image Define the image to create the volume.

1.2.3 Container Configuration File

NOTE: Starting in ACS Nexus v2.0.0, the container configuration file is not supported. The configuration file can be placed into a volume which is attached to a specific container.

1.2.4 Workflow

NOTE 1: Make sure the application is running before starting production from the Prober/Handler side, otherwise there could be data lost at the beginning of lot. Refer to the Nexus GUI for application status.

NOTE 2: Do not stop or restart Nexus during SmarTest session, otherwise Applications on the Edge server cannot be destroyed automatically. This will result in an exception in the deployment of Applications on the Edge server the next time SmarTest is started.

Following is a diagram that illustrates workflow of the Application on the ACS Edge Server.

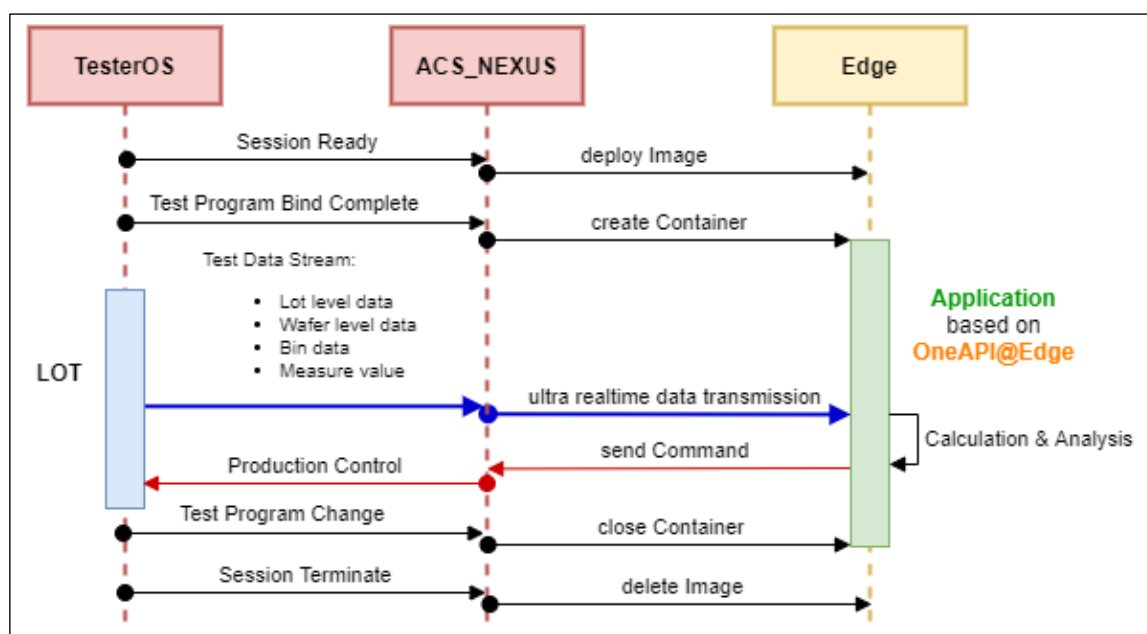


Figure Error! No text of specified style in document.-2. Application Workflow on ACS Edge Server

1.3 ACS Nexus GUI

NOTE: ACS Nexus automatically starts and runs perpetually on the Host Workstation after installation. If an ACS Nexus restart is needed, see [Restarting ACS Nexus](#).

ACS Nexus provides a graphic interface to present the system status, command records, and exception information, all of which can be automatically started and closed according to ACS Nexus configuration. The ACS Nexus GUI consists of a System Status area that provides connection status information for all connected systems, and an Information area that provides command records and exception information, as indicated in the figure below.

NOTE: Only one user at a time can run ACS Nexus on the same Host Workstation.

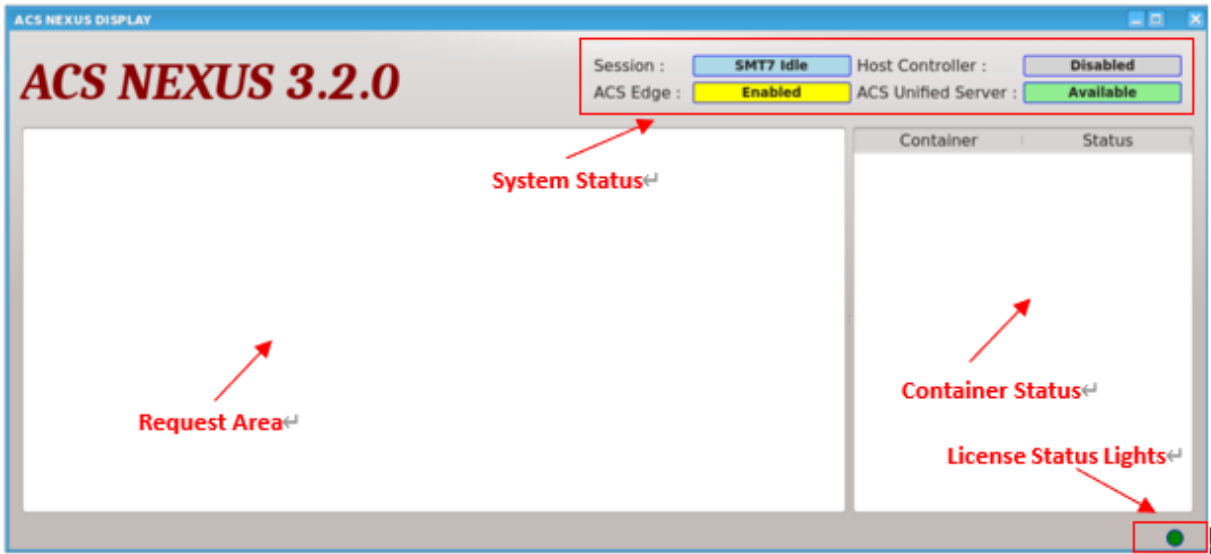


Figure Error! No text of specified style in document.-3. ACS Nexus GUI

License availability information is displayed by circular indicator in the lower right of the window. This indicator represents data collection license availability. Green indicates the license is available, red indicates that the license is either missing or invalid, and grey indicates that the license is not checked.

For additional details on the ACS Nexus GUI, see [Table Error! No text of specified style in document.-6](#) and [Table Error! No text of specified style in document.-7](#).

Table Error! No text of specified style in document.-6. System Status Indicators

System	Status Indicator
Session	Ready SmarTest Session has been started. Shows the SmarTest version (SMT7 or SMT8). Idle SmarTest Session has been already terminated. Shows the SmarTest version (SMT7 or SMT8)
Host Controller	Disabled ACS Nexus does not provide connection service to the application based on OneAPI at the Host Controller. Initial Fail ACS Nexus failed to provide connection service to the application based on OneAPI at the Host controller. Disconnected ACS Nexus is ready to provide connection service to the application based on OneAPI at the Host controller, but no application is connected at present. Connected ACS Nexus is ready to provide connection service to the application based on OneAPI at the Host controller, and the application is currently connected.
ACS Edge Server	Disabled ACS Nexus is configured not to provide service to ACS Edge Server. Enabled ACS Nexus is configured to provide service to ACS Edge Server. Down ACS Nexus detects that ACS Edge Server is not available. Available ACS Nexus detects that ACS Edge Server is available, and ACS Nexus is ready to provide service to the Edge Server.
ACS Unified Server	Enabled ACS Nexus is configured to monitor connection status to the ACS Unified Server. No Monitoring ACS Nexus is configured not to monitor connection status to ACS Unified Server. Not Available ACS Nexus is configured to monitor connection status to the ACS Unified Server, but the ACS Unified Server is not presently responding. Available ACS Nexus is configured to monitor connection status to the ACS Unified Server, and the ACS Unified Server is currently responding.

Table Error! No text of specified style in document.-7. Information Area Contents

Item	Status Indicator
Record of Control Information*	This area will record the information of control commands triggered by the application. Each line presents a command with the following information, separate by " ". For example: <pre>2023-07-15 10:00:15 Edge DebugApp_ADV_01 at 2023-07-15 10:00:15 PAUSE just test for Pause Command sent from Edge</pre> <pre>2023-07-15 10:00:30 Edge DebugApp_ADV_01 at 2023-07-15 10:00:30 STOP just test for Stop Command sent from Edge</pre> Received time The timestamp when ACS Nexus received this command. Resource The resource of the command, either Edge or TestFloor_Server Application The application information in the form of: {Name}_{Vendor}_{Version}. Sent time The timestamp when the application sent this command. Command The name of command, either Pause or Stop. Reason The reason why application sends this command. There is no limit to the text format.
Exception Information*	This area will print the exception information related to dynamic switch. Each line presents an exception with the following information starting with "Warning." For example: <pre>Warning 2023-06-08 14:56:44 Edge -> Edge connection is enabled, but no images are configured</pre> Received time The timestamp when ACS Nexus threw this exception. Resource The resource of the exception (switch container). Reason The reason why application throws this exception. There is no limit to the text format.
Container Status Information	The container name and status information is displayed in a table. Hover over the container name to display the image:tag. Image-related status or container status is displayed as described below: pullStarting ACS Nexus is starting to pull image of target application. pullSuccess ACS Nexus pulls image of target application successfully. pullFailed ACS Nexus image pull of the target application failed. pullTimeout ACS Nexus pulls image of target application timeout. ImageDeleteStarting ACS Nexus is starting to delete the image of target application. imageDeleteSuccess

Item	Status Indicator
	ACS Nexus deleted image of the target application successfully. imageDeleteFailed ACS Nexus failed to delete image of the target application. creating ACS Nexus is starting to create the target application container. created ACS Nexus creates target application container successfully and container has never started since creation. createFailed ACS Nexus failed to create target application container. runStarting ACS Nexus is starting to run target application container. running The target application container is running. runFailed The target application container run failed. stopStarting ACS Nexus is starting to stop target application container. stopFailed ACS Nexus failed to stop target application container. removeStarting ACS Nexus is starting to remove target application container. removed The target application container is removed. removeFailed Removal of the target application container failed. restarting The target application container is in the process of restarting. paused The target application suspends all the processes for an indefinite time. exited The target application process inside the container terminated. dead This state is achieved when trying to remove the target application container, but it cannot be removed because some resources are still in use by an external process. removing The target application container is being removed. unknown An error occurred while getting the status.
License Status	Indicates the current license status. The status light will show one of three states: Green - indicates that the ACS Nexus DataCollection license is valid Red - indicates that the ACS Nexus DataCollection license is invalid. Grey – indicates that the ACS Nexus DataCollection license is not checked.

Item	Status Indicator
Certificate Expiration Reminder	When the GUI is launched, ACS Nexus will check the validity of the certificate. No prompt will be given if the usable period is greater than seven days. If the usable period is less than seven days or the certificate has expired, this area will print the exception information related to certificate status, with following information: Received time The timestamp when ACS Nexus threw this exception. Reason The reason why ACS Nexus threw this exception. There is no limit to the text format. Certificate Expiration Examples: Warning 2024-01-29 14:52:05 Certificate will be out of date in 2 days Warning 2024-01-29 14:58:41 Certificate has expired. Please renew the certificate NOTE: Certificate will be renewed when Nexus is restarted or SMT is terminated. After each successful renewal, Nexus will receive a new certificate that is valid for 90 days by default. If certificate renew fails, the previous certificates will not be overwritten or modified. To prevent service or runtime issues and eliminate the need for operators to manually restart the Nexus process, try to avoid long-term inactivity or extended uptime that surpasses the default 90-day certificate validity period.

*This area will be cleared in the event of SmarTest startup or ACS Nexus reboot.